

前期准备



1. 由于是教学示例，需求分析、模块划分这些**流程不会做的非常严谨，仅供参考**
2. 请按验收方案实现基础模块下的**基本功能**（黑色）；验收方案要求的**进阶功能**会用**红色**标记

需求分析

用户功能

1. 用户注册和登录
2. 用户信息管理
3. 第三方登录（Github）
4. 数据埋点与用户仪表盘
5. 头像管理与云对象存储

博客功能

1. 博客的创建、展示、搜索
2. 支持markdown等格式的文本编辑器
3. 博客热度排行榜
4. 博客的全文检索
5. 博客的语义检索（可以与4合并成混合检索）
6. 在线阅读人数统计
7. 博客推荐或者同风格用户推荐

点赞功能

1. 用户对博客和评论能进行点赞或取消点赞 ‘

评论功能

1. 博客评论
2. 划线评论

3. 引用评论（楼中楼）

订阅功能

1. 订阅与消息通知
2. 每日新闻/大会关注推送

日志功能

1. 日志和日志分析
2. 可观测性

运维

1. Docker compose编排
2. CI/CD

AI自动化方向

1. 文章内容总结
2. AI写作助手
3. Github Trending 自动解读
4. AI 技术论文 Digest

其它

1. 移动端适配
2. MCP Server
3. 任务栏与知识树
4. 话题讨论/留言墙
5. 漂流瓶
6. 小桌宠
7. 仿<https://neal.fun/>（感谢  怀玺 分享的网站，也可以学习这个网站展示大家的小实验和小项目）

数据库模型

你要根据自己想要实现的业务逻辑自行设计数据模型



题外话：由于不同数据库的sql写法存在差异，大家可以尝试<https://dbdiagram.io/>通过DBML定义模型，并按照实际需求导出特定数据库的sql语句

DBML示例：

代码块

```
1 Enum like_target_type {
2   post
3   comment
4 }
5
6 Table likes {
7   id integer [primary key, increment]
8
9   user_id integer [not null, note: '点赞用户ID']
10
11   target_type like_target_type [not null, note: '点赞对象类型: post 或 comment']
12   target_id integer [not null, note: '点赞对象ID, 对应 posts.id 或 comments.id']
13
14   created_at timestamp [not null, default: `now()`]
15
16   indexes {
17     (user_id, target_type, target_id) [unique]
18     (target_type, target_id)
19   }
20 }
```

一些tips:

1. 由于点赞对象可以是博客也可以是评论，因此可以创建一个枚举型，后面也方便扩展新的点赞对象
2. 根据需求，我们要**创建合适的索引以加速查询**。例如我们要判断一个用户是否已经给该对象点过赞了，那么就创建(user_id, target_type, target_id)联合索引查询



推荐回顾知识图谱中的**数据库索引**部分

设计API



推荐回顾知识图谱中的**API设计**部分

接下来要根据具体业务设计前后端通信所需的API

代码块

```
1 // 统一response, 之后的response都是对data的描述
2 {
3     "code": 0, // 错误码, 0表示成功, 其它可以参照网上的一些俗成约定
4     "msg": 'success', // 返回信息
5     "data": any // 返回数据
6 }
```

用户注册和登录为例:

URL: {host}/api/v1/auth/register

Method: POST

代码块

```
1 //我的注册逻辑涉及这三个
2 {
3     "username": "alice",
4     "email": "alice@example.com",
5     "password": "password123"
6 }
```

代码块

```
1 //再次声明这是"data"部分
2 {
3     "user": {
4         "id": 1,
5         "username": "alice",
6         "email": "alice@example.com",
7         "created_at": "2026-05-25T10:00:00Z"
8     },
9     "access_token": "jwt-token"
10 }
```

项目结构



根据你的语言设计合理的项目结构和目录树

如果你已经跟随知识图谱对代码规范进行了一定的探索, 那么你应该对一些架构已经有所了解, 例如 MVC、DDD等

教学示例主要依照了MVC的思想，划分了三层：

1. Controller层，主要解析请求，返回响应
2. Service层，具体的业务逻辑
3. DAO层(Data Access Object) ，在这里主要是用ORM与数据库交互

(**Repository**要比DAO要更上层一点，可能隐藏了选择内存、本地文件、缓存或者数据库这种细节，即可能使用了多种DAO。在这里出于实际需求和代码量的考虑，只用了一种DAO，但其实也简单地多封装了一层为Repository)

后端目录树

代码块

```
1  backend/
2  |—— cmd/                                # 程序入口, main.go
3  |   |—— api/
4  |   |   |—— main.go                    # HTTP API 启动入口
5  |   |—— mcp/
6  |       |—— main.go                    # MCP server启动入口
7  |
8  |—— api/                                # 第三方API
9  |   |—— openapi/
10 |       |—— helloblog.v1.yaml
11 |
12 |—— etc/
13 |   |—— config.example.yaml            # 配置文件
14 |
15 |—— internal/
16 |   |—— config/
17 |       |—— config.go                  # 配置加载
18 |   |
19 |   |—— server/
20 |       |—— http.go                    # Gin server 装配
21 |       |—— router.go                  # 总路由注册
22 |       |—— user_routes.go             # 用户模块路由注册
23 |       |—— middleware/                # Gin 中间件
24 |           |—— auth.go                # JWT 鉴权
25 |           |—— cors.go                # 跨域处理
26 |       |
27 |       |—— svc/
28 |           |—— service_context.go      # 参考 go-zero, 将 infra、controller、
service 统一装配
29 |       |
30 |       |—— controller/                # Controller 层
31 |           |—— user/
```

```

32 | | | | └─ controller.go      # 用户 Controller 结构体与依赖
33 | | | | └─ register.go      # 用户注册接口
34 | | | | └─ login.go        # 用户登录接口
35 | | | | └─ me.go            # 当前用户信息接口
36 | | | └─ post/
37 | | |   └─ ...              # 文章模块接口
38 | | └─ ...                  # comment、like、search 等模块接口
39 |
40 | └─ dto/                   # DTO: 前后端通信的请求/响应对象
41 |   └─ user.go              # 用户 DTO
42 |   └─ ...                  # post、comment、like、search 等 DTO
43 |
44 | └─ service/               # Service 层
45 |   └─ user/
46 |     └─ service.go         # 用户Service
47 |     └─ register.go        # 用户注册业务逻辑
48 |     └─ login.go           # 用户登录业务逻辑
49 |     └─ me.go               # 当前用户查询业务逻辑
50 |     └─ mapper.go           # 用户模型转换
51 |   └─ post/
52 |     └─ ...                  # 文章业务逻辑
53 |   └─ comment/
54 |     └─ ...                  # 评论业务逻辑
55 |   └─ ...
56 |
57 | └─ dao/                   # DAO 层
58 |   └─ model/               # GORM Model, 要与 migrations 中的表结构对
    齐
59 | | | | └─ user.go           # 用户表模型
60 | | | | └─ ...
61 | | | └─ user.go           # 用户表数据访问
62 | | |   └─ ...
63 |
64 | └─ infra/                 # 基础设施: 数据库、Redis、本地模型客户端
65 |   └─ db/
66 |     └─ postgres.go        # PostgreSQL 和 GORM 初始化
67 |     └─ transaction.go     # 数据库事务工具
68 |   └─ redis/
69 |     └─ redis.go            # Redis 初始化
70 |   └─ embedding/
71 |     └─ client.go           # 本地词嵌入模型客户端
72 |
73 | └─ pkg/                   # 这里是一些与业务逻辑关系不大、比较通用容易复
    用的代码
74 |   └─ response/             # 统一响应结构、错误码
75 |   └─ jwt/                  # JWT 签发与校验
76 |   └─ password/             # 密码哈希

```

```
77 |      └─ ...                                # pagination、validator、logger 等通用工具
78 |
79 | └─ migrations/
80 |   └─ 000001_init_schema.sql                # 初始化 SQL 语句
81 |
82 | └─ go.mod
```